

Análise de Segurança

Levantamento do estado atual do projeto e dos pontos que faltam endereçar antes de tratar a aplicação como apta a proteger dados sensíveis.

Repositório: `sescacre/controle_impressoes`

Data da análise: 08/10/2026 · **última atualização:** 09/10/2026

Escopo: servidor Node (`server.js` / `server/api.js` / `server/db.js`), front-end (ts/), Docker, dependências e configuração de banco

Resumo dos achados

1	CRÍTICO	API sem autenticação/autorização no servidor
2	CRÍTICO	XSS armazenado via <code>innerHTML</code> sem escape
3	ALTO	Credencial do admin exposta no bundle público
4	ALTO	MySQL root com senha em branco, aberto para faixas de IP amplas
5	MÉDIO	Sem HTTPS visível na aplicação
6	MÉDIO	Nenhum security header (CSP, X-Frame-Options, etc.)
7	MÉDIO	Bibliotecas via CDN sem Subresource Integrity (SRI)
8	BAIXO	Vulnerabilidade transitiva em devDependency (<code>dompurify/jspdf</code>)
9	BAIXO	Sem rate limiting em nenhuma rota
10	BAIXO	Dados internos versionados no Git — confirmar repositório privado

● Crítico

PrintGest — Análise de Segurança — gerado em 08/10/2026 — atualizado em 09/10/2026

CRÍTICO 1. A API não tem nenhuma autenticação ou autorização no servidor

O login da "Área Administrativa" é **inteiramente do lado do cliente** — `ts/ui/admin.ts` compara usuário e senha em JavaScript, no navegador. O backend (`server/api.js`) aceita `GET` , `PUT` , `PATCH` em `/api/db/*` e `POST /api/import` **sem checar sessão, token, cookie ou qualquer credencial**.

Por que importa: qualquer pessoa com acesso de rede ao servidor pode ler e **sobrescrever** equipamentos, leituras e meses direto via `curl` /Postman, sem nunca passar pela tela de login — inclusive foi assim que os dados históricos de 2024/2025 foram importados nesta mesma sessão de trabalho, via requisição HTTP direta ao endpoint.

Correção sugerida: mover a checagem de usuário/senha para o servidor (ex.: sessão com cookie `HttpOnly` ou `JWT`), e exigir essa sessão válida em toda rota de escrita (`PUT` / `PATCH` / `POST /api/import`) dentro de `server/api.js` .

Atualização (09/10/2026): criada a tabela `usuarios` (com a conta `admin` semeada) e a rota `POST /api/usuarios` , que grava novas contas com senha sempre hashada em `bcrypt` — base para o login real no servidor. Isso **ainda não fecha o achado**: o login do cliente continua validando contra `ADMIN_USER` / `ADMIN_PASS` sem tocar nessa tabela, e `POST /api/usuarios` em si também está sem autenticação — hoje qualquer um pode criar uma conta admin direto nessa rota. Falta o passo de sessão/login server-side para resolver de fato.

CRÍTICO 2. XSS armazenado (stored XSS), consequência direta do item 1

`ts/ui/tables.ts` (e outros módulos de UI) inserem campos vindos do banco (`sigla` , `setor` etc.) diretamente via `innerHTML` , sem escapar o conteúdo.

Por que importa: como qualquer um pode escrever nesses campos (item 1), um atacante pode gravar algo como `<img src=x onerror=...>` num nome de setor, e esse código executa no navegador de **todo usuário** que abrir o dashboard — permite roubo de sessão, redirecionamento, desfiguração da página.

Correção sugerida: escapar qualquer valor vindo do banco antes de inserir no DOM (função de escape de HTML, ou trocar `innerHTML` por `textContent` /criação de nós DOM nos campos de dados). Fazer isso mesmo depois de corrigir o item 1, como defesa em profundidade.

● Alto

ALTO 3. Credencial do admin exposta no bundle público

PrintGest — Análise de Segurança — gerado em 08/10/2026 — atualizado em 09/10/2026

`ADMIN_USER` / `ADMIN_PASS` (`ts/config.ts`) vão em texto puro para o `dist/app.js` que qualquer visitante baixa — "ver código-fonte" da página revela a senha. Já documentado como limitação conhecida no README do projeto, mas vale destacar que não é apenas teórico: a senha atual está literalmente no repositório e no bundle publicado.

Correção sugerida: só existe correção real junto com o item 1 (autenticação no servidor) — nenhuma credencial deve ficar acessível no código que roda no navegador.

ALTO 4. MySQL com usuário root e senha em branco, liberado para faixas de IP amplas

A aplicação conecta ao MySQL como `root`, sem senha. Durante os ajustes de deploy desta conversa, esse acesso foi liberado (`GRANT`) para as faixas `192.168.5.%` e o host `192.168.0.101`.

Por que importa: qualquer máquina dessas faixas — ou que comprometa uma delas — tem controle total do servidor MySQL inteiro, não apenas do banco `controle_impressoes`.

Correção sugerida: criar um usuário de banco dedicado à aplicação, com senha forte e privilégios restritos apenas ao banco `controle_impressoes` (sem `GRANT global`), e revogar o acesso root remoto assim que possível.

● Médio

MÉDIO 5. Sem HTTPS visível na aplicação

`server.js` usa `http.createServer` puro, sem TLS. Não foi possível confirmar neste levantamento se o Dokploy/Traefik está terminando HTTPS na frente do container em produção — isso precisa ser verificado diretamente na configuração de domínio/proxy do deploy.

MÉDIO 6. Nenhum security header configurado

Faltam cabeçalhos como `Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options` e `Strict-Transport-Security`. Hoje nada limita clickjacking, MIME-sniffing, ou reduz o estrago de um eventual XSS (item 2).

MÉDIO 7. Bibliotecas de terceiros via CDN, sem Subresource Integrity (SRI)

PrintGest — Análise de Segurança — gerado em 08/10/2026 — atualizado em 09/10/2026

Chart.js, jsPDF, SweetAlert2 e SheetJS/xlsx são carregados em runtime via `cdnjs / jsdelivr` (`ts/config.ts`, `ts/utils/scriptLoader.ts`), com versões fixadas mas sem atributo `integrity`.

Por que importa: se um desses CDNs for comprometido, o código malicioso injetado roda com acesso total à página (e, combinado com o item 1, ao banco de dados).

Correção sugerida: adicionar hashes SRI (`integrity` + `crossorigin`) nos `<script>` carregados dinamicamente, ou empacotar essas bibliotecas localmente no build.

Baixo / informativo

BAIXO 8. Vulnerabilidade transitiva crítica reportada pelo `npm audit`

`npm audit` aponta uma vulnerabilidade crítica em `dompurify`, puxada pela devDependency `jspdf`. O jsPDF realmente usado em produção vem fixado via CDN (versão 2.5.2), não do `node_modules` — o impacto prático hoje é só na ferramenta de build/typecheck, não em produção. Ainda assim, vale conferir se a versão 2.5.2 do CDN carrega CVEs equivalentes.

BAIXO 9. Nenhum rate limiting em nenhuma rota

Hoje é um ponto secundário, já que não existe login real no servidor — mas passa a ser importante assim que o item 1 for implementado, para evitar força bruta contra a autenticação.

BAIXO 10. Dados internos versionados no Git

`db/schema.sql` (inventário de equipamentos e histórico real de leituras) está versionado no repositório `sescacre/controle_impressoes` no GitHub. Recomenda-se confirmar diretamente no GitHub se esse repositório está marcado como privado.